

Assignment 5—OOP and Exceptions

Due: 21 May

We want to write some Ascii art classes that draw rectangular artworks by printing characters instead of using graphics. For example, a triangle art program might print something like this:

```
*  
**  
***  
****  
*****
```

For this to work, your terminal or console program has to use a monospaced font (also known as a typewriter font, where every character has the same width, like *Courier New*.)

This is a good candidate for object-oriented programming, as all artworks will share a common interface and we want to be able to use any art-drawing class in any program that is coded to the interface.

I have designed and implemented a java interface class, `AsciiArt.java`, as well as an abstract base class `AsciiArtBase.java` that is intended to make it easy to write art-drawing subclasses.

We want clients to be able to specify the number of rows and columns in the “canvas”, so the art classes must be prepared to execute their drawing algorithm correctly on a canvas of any size.

I have posted the code along with the assignment.

Here is the interface class `AsciiArt.java`:

```
/*  
    AsciiArt.java  
    Last modified by J.Finn: 9-May-2026  
  
    Interface for ASCII art programs that draw character-based  
    art over any rectangular grid.  
  
    It is expected that the initial number of rows and columns  
    will be AsciiArt constructor arguments, but they are intended  
    to be modifiable to produce variations on the art of  
    different sizes.  
*/
```

```

public interface AsciiArt
{
    /*
     * Print a description of the art
     */
    public void describe();

    /*
     * Returns the number of rows in the canvas
     */
    public int getRows();

    /*
     * Sets the number of rows in the canvas
     * Returns: true normally, false if the requested
     * number of rows is inappropriate.
     */
    public boolean setRows(int rows);

    /*
     * Returns the number of columns in the canvas
     */
    public int getCols();

    /*
     * Sets the number of columns in the canvas.
     * Returns: true normally, false if the requested
     * number of columns is inappropriate.
     */
    public boolean setCols(int cols);

    /*
     * Draws the board to standard output on getRows()
     * lines of length getCols().
     * Return true normally, false on error.
     */
    public boolean draw();

    /*
     * Returns a newly-created two-dimensional array of char,
     * which will have getRows() rows and getCols() columns,
     * and which contains the same drawing that would be printed
     * to standard output by calling draw().
     * Arrays are mutable, so this must be a new array, not a
     * reference to any private implementation data structure.
     * In other words, if the client modifies this array, it
     * should not affect the array returned by subsequent calls
     * to artwork() or the art drawn by subsequent calls to
     * draw().
     * Returns null on error.
     */
    public char [] [] artwork();
};

```

AsciiArtBase is abstract. The core drawing function that will be different for every game is **artwork**. AsciiArtBase implements **draw** by calling artwork, so individual games don't have to write **draw**. As we recommended, the base class has instructions for how to write subclasses:

```

/*
    AsciiArtBase.java
    Last modified by J.Finn: 9-May-2026

    Abstract base class.

    To create an art work:
    a) create a subclass of ArtBase
    b) The constructor should initialize rows, cols, and
       description
    c) Implement the artwork() function
    d) If you do not wish to allow all values for rows and cols,
       you optionally may override getRows and setRows.
*/

public abstract class AsciiArtBase implements AsciiArt
{
    protected String description; // artwork description
    protected int rows;          // current number of canvas rows
    protected int cols;          // current number of canvas columns

    /*
        Print a description of the art
    */
    public final void describe()
    {
        System.out.println(description);
    }

    /*
        Returns the number of rows in the canvas
    */
    public final int getRows()
    {
        return rows;
    }

    /*
        Sets the number of rows in the canvas
        Returns: true normally, false if the requested
        number of rows is inappropriate.
    */
    public boolean setRows(int rows)
    {
        if (rows < 1) return false;
        this.rows = rows;
        return true;
    }

    /*
        Returns the number of columns in the canvas
    */
    public final int getCols()
    {
        return cols;
    }
}

```

```

    /*
        Sets the number of columns in the canvas
        Returns: true normally, false if the requested
        number of columns is inappropriate.
    */
    public boolean setCols(int cols)
    {
        if (cols < 1) return false;
        this.cols = cols;
        return true;
    }

    /*
        Draws the board to standard output on getRows() lines of
        length getCols().
        Return true normally, false on error.
    */
    public final boolean draw()
    {
        char [] [] canvas = artwork();
        if (canvas == null) return false;
        for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < cols; j++)
                System.out.print(canvas[i][j]);
            System.out.println();
        }
        return true;
    }

    /*
        Returns a newly-created two-dimensional array of char,
        which will have getRows() rows and getCols() columns,
        and which contains the same drawing that would be printed
        to standard output by calling draw().
        Arrays are mutable, so this must be a new array, not a
        reference to any private implementation data structure.
        In other words, if the client modifies this array, it
        should not affect the array returned by subsequent calls
        to artwork() or the art drawn by subsequent calls to
        draw().
        Returns null on error.
    */
    public abstract char [] [] artwork();
};

```

The posted code includes two artwork classes. One of them is the above triangle-drawing class:

```

public class TriangleArt extends AsciiArtBase
{
    private String desc =
        "This art prints a triangle that fills the lower-left half of the canvas";
    private char c;

    /*
        rows, cols are the initial rows and columns
        of the canvas.
        c is the char to be used in drawing
    */
    public TriangleArt(int rows, int cols, char c)
    {
        if (rows >= 1) this.rows = rows;
        else this.rows = 10;
        if (cols >= 1) this.cols = cols;
        else this.cols = 10;
        this.description = desc;
        this.c = c;
    }

    public char [] [] artwork()
    {
        char [] [] art = new char[rows][cols];
        double ratio = (double) rows / cols;
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                if (i >= ratio * j) art[i][j] = c;
                else art[i][j] = ' ';
        return art;
    }
};

```

Subclasses must in their constructors initialize relevant variables such as the number of rows and columns. Otherwise, the only function that must be implemented by subclasses is **artwork**.

There is also an ArtDemo.java class with a main program that demonstrates working with AsciiArt classes.

Problem 1

The AsciiArt classes use function error return values to indicate errors. **setRows** and **setCols** return false if the client tries to set a number of canvas rows or columns that are inappropriate for the artwork, which should document what canvas sizes are acceptable. **artwork** returns null on error and **draw** returns false on error.

Modify the interface and implementation to throw exceptions instead of using error return values. Follow the guidelines from the handout on exceptions: design a class hierarchy for ArtException errors with a base class and with subclasses for the specific error types: size error or drawing error. The boolean functions just mentioned can be revised to have return type **void**, as errors will be reported by throwing exceptions. Revise the comments that document how the functions work to reflect the changes you make.

Problem 2

Write two artwork subclasses that draw whatever you like. You may not rewrite `TriangleArt` or `Alternator`; you must design your own classes. Use your solutions for Problem 1 as the interface and base class. Also, write a demo class with a main program that illustrates that your art-drawing classes work correctly and that the exception mechanisms work correctly.

Art classes that demonstrate significant design and implementation effort *may*, at the sole discretion of the grader, receive extra credit.

Make sure your name is in a comment at the top of each java source file.

Submit a zip file that contains all of your java source files.